

SLOVENSKÁ TECHNICKÁ UNIVERZITA
V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Tímový projekt

Virtual Data Analyst

Inteligentný systém pre analýzu dát pomocou prirodzeného
jazyka

Autori:	Peter Muzslay Janka Tarčáková Jozef Guláš Jana Kampová Tamara Uhrinová
Vedúci projektu:	Adam Kňaze
Externý zadávateľ:	PwC
Akademický rok:	2025/2026

Bratislava, 2026

Zadanie tímového projektu

Opis zadania

Cieľom tímového projektu je návrh a implementácia aplikácie **Virtual Data Analyst**, ktorá využíva generatívnu umelú inteligenciu na sprístupnenie dátovej analýzy netech-nickým používateľom. Aplikácia poskytuje jednoduché webové používateľské rozhranie, v ktorom používateľ kladie otázky týkajúce sa dát v prirodzenom jazyku, napríklad otázky o najziskovejšej oblasti služieb, počte záznamov s chýbajúcimi hodnotami alebo výskyte anomálií v transakčných dátach.

Systém používa generatívnu umelú inteligenciu na porozumenie otázke, identifikáciu relevantných dátových zdrojov a vytvorenie vyhľadávacieho alebo databázového dotazu, ktorým sa získajú potrebné údaje z firemných databáz. Pri otázkach, ktoré nie sú vhodné ako čistý databázový dotaz, systém využíva preddefinované analytické moduly, napríklad na detekciu anomálií, kontrolu chýbajúcich hodnôt alebo iné formy dátového spracovania.

Na základe získaných výsledkov aplikácia vytvorí zrozumiteľnú odpoveď pre používa-teľa a zobrazí ju vo webovom rozhraní. V prípade vhodného typu výstupu môže odpoveď doplniť aj vizualizáciou dát. Projekt je realizovaný ako externé zadanie pre spoločnosť PwC a je zameraný na praktické využitie moderných GenAI nástrojov v oblasti podnikovo-vej dátovej analytiky.

Obsah

Zadanie tímového projektu	II
1 Úvod	1
1.1 Motivácia	1
1.2 Štruktúra projektu	1
2 Riešený problém	2
2.1 Definícia problému	2
2.2 Príklad použitia	2
3 Prístup a metodológia	3
3.1 Výber technológií	3
3.2 Architektúra GenAI Core	3
3.2.1 Query Generator	3
3.2.2 Response Summarizer	4
4 Architektúra systému	5
4.1 Vysokoúrovňová architektúra (High-Level)	5
4.1.1 Tok dát (Data Flow)	5
4.2 Technická architektúra (Low-Level)	6
4.2.1 Backend komponenty	7
4.2.2 Router: LLM alebo preddefinovaný modul	7
4.2.3 API Endpoints	8
4.2.4 Bezpečnostné mechanizmy	8
4.2.5 Frontend komponenty	9
5 Frontendová časť aplikácie	10
5.1 Štruktúra projektu	10
5.2 Architektúra a dátový tok	10
5.3 Správa stavu (State Management)	10
5.4 Komunikácia s backendom	11
5.5 Rozbor kľúčových UI komponentov	11
5.5.1 Komponent Chat	11
5.5.2 Komponent SideBar (Navigačný panel)	12
5.5.3 Komponent DatabaseModal	13
5.5.4 Komponent ChatHeader	14
6 Výber a testovanie LLM modelov	15
6.1 Metodológia testovania	15
6.2 Porovnávané modely	15
6.3 Výsledky testovania - SQL generácia	15
6.4 Výsledky testovania - Sumarizácia	16
6.5 Záver výberu modelov	16

7	Použité technológie - detailný prehľad	17
7.1	Backend technológie	17
7.2	Frontend technológie	18
7.3	Infraštruktúra	19
7.4	AI/ML modely	19
8	Databázová vrstva	20
8.1	Podporované databázové systémy	20
8.2	Správa databázového pripojenia	20
8.2.1	Získavanie schémy	20
8.2.2	Vykonávanie dotazov	20
8.3	READ-ONLY prístup a bezpečnosť	21
8.4	Parametrizácia pripojenia	21
8.5	API endpointy pre správu pripojenia	22
8.6	Testovacie databázy a Docker Compose	22
8.7	MongoDB - história chatových relácií	22
8.8	Zachovanie kontextu medzi otázkami	23
8.8.1	Kontext pre SQL generátor	23
8.9	Redis - cachovanie	24
8.9.1	Sémantická Q&A cache	25
8.9.2	Cache metriky a diagnostika	25
9	CI/CD a nasadenie	27
9.1	Backend CI	27
9.2	Frontend CI	27
9.3	CD na Azure VM	27

1 Úvod

Projekt **Virtuálny dátový analytik** predstavuje moderný webový systém, ktorý umožňuje používateľom analyzovať dáta v databázach pomocou prirodzeného jazyka. Využitím pokročilých techník spracovania prirodzeného jazyka (NLP) a veľkých jazykových modelov (LLM) dokáže systém transformovať otázky v bežnom jazyku na SQL dotazy, vykonať ich nad databázou a následne vrátiť zrozumiteľné odpovede.

Hlavným cieľom projektu je demokratizácia prístupu k dátovej analytike - umožniť aj netechnickým používateľom efektívne získavať informácie z databáz bez potreby znalosti SQL alebo iných programovacích jazykov.

1.1 Motivácia

V súčasnej dobe je dátová analytika kľúčovým faktorom pri rozhodovaní v mnohých oblastiach podnikania. Avšak tradičné nástroje pre prácu s databázami vyžadujú:

- Znalosť SQL jazyka
- Pochopenie štruktúry databázy
- Technické zručnosti pre interpretáciu výsledkov

Virtuálny dátový analytik tieto bariéry odstraňuje a sprístupňuje dátovú analýzu širšiemu okruhu používateľov.

1.2 Štruktúra projektu

Projekt je rozdelený do nasledujúcich hlavných komponentov:

1. **Frontend** - Používateľské rozhranie (React + Vite)
2. **Backend** - Serverová aplikácia (FastAPI, Python)
3. **GenAI Core** - Jadro pre spracovanie prirodzeného jazyka (LLM)
4. **Databázová vrstva** - Pripojenie a správa databáz

2 Riešený problém

2.1 Definícia problému

Hlavný problém, ktorý projekt rieši, je **preklad prirodzeného jazyka na štruktúrované databázové dotazy** a následná **interpretácia výsledkov** pre koncového používateľa.

Konkrétne výzvy zahŕňajú:

- **Sémantické porozumenie** - Pochopenie zámeru používateľa z neformálnej otázky
- **Mapovanie na schému** - Správne priradenie entít v otázke k tabuľkám a stĺpcom databázy
- **Generovanie validného SQL** - Syntakticky a sémanticky správne dotazy
- **Bezpečnosť** - Zabezpečenie len čítacích operácií (READ-ONLY)
- **Zrozumiteľná odpoveď** - Sumarizácia výsledkov v prirodzenom jazyku

2.2 Príklad použitia

Vstup (používateľ)	Výstup (systém)
„Koľko objednávok bolo vytvorených v novembri 2019?“	Vygenerovaný SQL dotaz, počet záznamov a textová sumarizácia výsledkov
„Aký je priemerný počet položiek na objednávku?“	Analytický výsledok s vysvetlením
„Ukáž mi top 10 produktov podľa predaja“	Zoradený zoznam s popisom

Tabuľka 1: Príklady interakcie s virtuálnym analytikom

3 Prístup a metodológia

3.1 Výber technológií

Pri návrhu systému sme zvolili nasledujúce technológie:

Komponent	Technológia	Zdôvodnenie
Frontend	React + Vite	Moderný, reaktívny framework s rýchlym vývojovým prostredím
Backend	FastAPI (Python)	Asynchrónne API s automatickou dokumentáciou
Databáza	PostgreSQL	Robustná open-source relačná databáza
LLM	GPT-4.1 / Gemini 2.5	Jazykové modely pre NLP úlohy
Cache	Redis	In-memory databáza pre rýchle ukladanie do vyrovnávacej pamäte
Kontajnerizácia	Docker	Konzistentné prostredie pre vývoj a nasadenie

Tabuľka 2: Použíte technológie a ich zdôvodnenie

3.2 Architektúra GenAI Core

Jadro systému pre spracovanie prirodzeného jazyka pozostáva z troch hlavných komponentov:

1. **Router (LLM)** - Rozhoduje o spôsobe spracovania dotazu
2. **Query Generator** - Generuje SQL dotazy z prirodzeného jazyka
3. **Response Summarizer** - Sumarizuje výsledky pre používateľa

3.2.1 Query Generator

Komponent využíva model GPT-4.1 pre generovanie SQL dotazov. Systémový prompt definuje pravidlá:

- Generovať len **SELECT** dotazy (read-only prístup)
- Nikdy nepoužívať INSERT, UPDATE, DELETE, DROP, CREATE a iné modifikačné príkazy
- Vrátiť len čistý SQL dotaz bez vysvetlení
- Používať správnu syntax podľa typu databázy

3.2.2 Response Summarizer

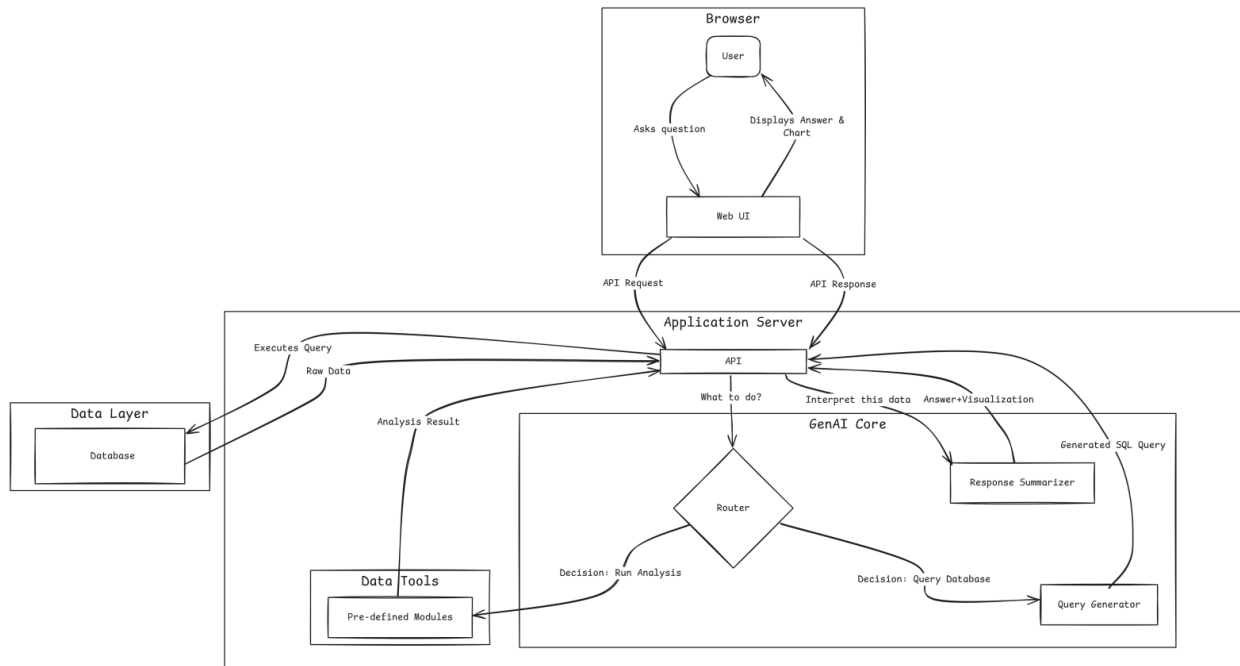
Komponent využíva model Gemini 2.5 Flash pre sumarizáciu výsledkov. Poskytuje:

- Popis, čo dáta zobrazujú
- Zvýraznenie kľúčových poznatkov a vzorcov
- Zmienku o významných hodnotách alebo trendoch
- Zrozumiteľný text pre netechnických používateľov

4 Architektúra systému

4.1 Vysokoúrovňová architektúra (High-Level)

Systém je navrhnutý ako **trojvrstvová architektúra** s jasne oddelenými zodpovednosťami:

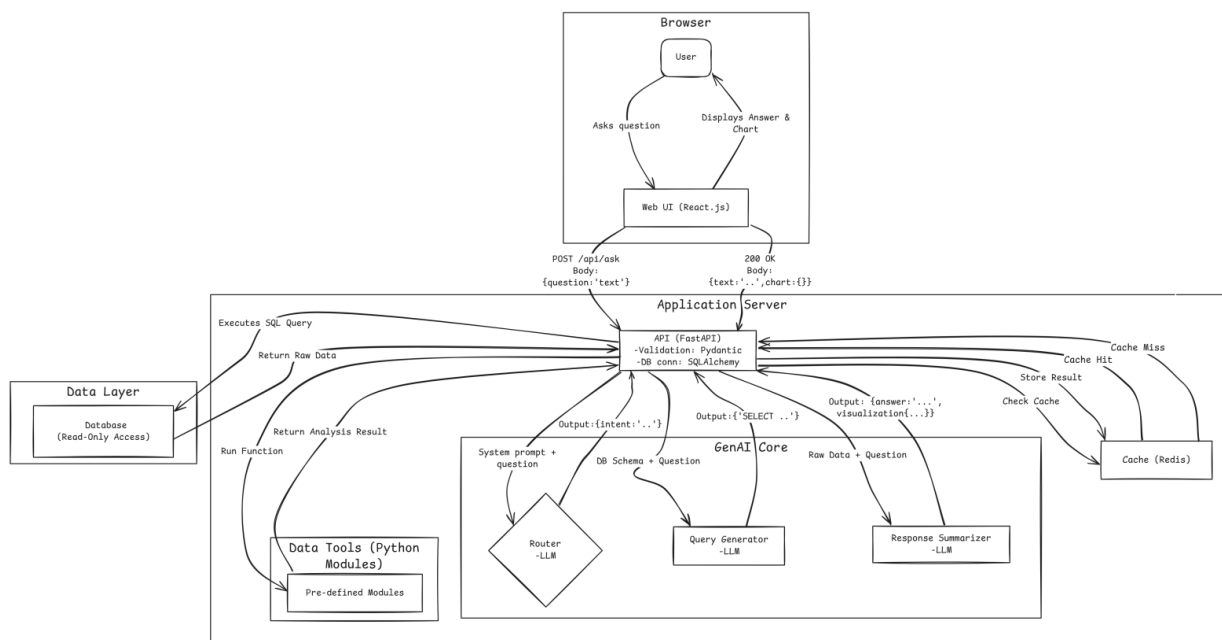


Obr. 1: Vysokoúrovňová architektúra systému

4.1.1 Tok dát (Data Flow)

1. Používateľ zadá otázku v prirodzenom jazyku cez webové rozhranie
2. Frontend odošle požiadavku na Backend API (/api/ask)
3. Backend získa schému databázy
4. Query Generator (GPT-4.1) vygeneruje SQL dotaz
5. Dotaz sa vykoná nad databázou (READ-ONLY)
6. Response Summarizer (Gemini 2.5) vytvorí textovú sumarizáciu
7. Odpoveď sa vráti používateľovi

4.2 Technická architektúra (Low-Level)



Obr. 2: Detailná architektúra systému s komponentmi

4.2.1 Backend komponenty

Komponent	Popis
DatabaseManager	Správa pripojenia k PostgreSQL, MySQL, Oracle, SQL Server; vynucovanie READ-ONLY módu; získavanie schémy
QueryGenerator	LLM-based generovanie SQL dotazov pomocou Azure GPT-4.1
ResponseSummarizer	LLM-based sumarizácia výsledkov pomocou Google Gemini 2.5 Flash
RedisClient	Multi-layer caching schémy, SQL queries, výsledkov a sumarizácií pre optimalizáciu
SemanticQACache	Semantic caching s vektorovými embeddingami a kosínusovou podobnosťou
EntraIDAuth	Validácia JWT tokenov z Azure Entra ID a bezpečnostný middleware
ChatHistory	Správa chat histórie v MongoDB s indexovaním a session manažmentom
MongoClient	Asynchrónne pripojenie a správa MongoDB databázy
ChartIntentDetector	Detekcia požiadavky na grafickú vizualizáciu a extrakcia parametrov grafu
ChartRenderer	Vygenerovanie PNG grafu na základe SQL výsledkov pomocou Matplotlib
Router	LLM-based rozhodovanie medzi Data Tools a Database queries

Tabuľka 3: Backend komponenty a ich zodpovednosti

4.2.2 Router: LLM alebo preddefinovaný modul

Backend spracováva otázky cez komponent **Router**, ktorý rozhoduje, či je vhodnejšia databázová cesta alebo analytický modul. Pri databázovej ceste sa otázka preloží na SQL dotaz, dotaz sa vykoná nad pripojenou databázou a výsledok sa sumarizuje. Pri analytickej ceste router odovzdá požiadavku modulom `data_tools`, ktoré riešia úlohy vhodnejšie pre deterministické Python spracovanie.

Motiváciou routera je skutočnosť, že nie každá analytická otázka je ideálna ako SQL dotaz. SQL je vhodné pre filtrovanie, agregácie, zoradovanie, spojenia tabuliek a výpočty, ktoré sa dajú jasne vyjadriť nad relačnou schémou. Naopak otázky typu detekcia anomálií, hľadanie chýbajúcich hodnôt, porovnanie distribúcií, štatistické testy alebo pokročilejšia transformácia dát môžu byť bezpečnejšie a presnejšie riešené preddefinovaným Python modulom. Takýto modul vie použiť knižnice pre dátovú analýzu, vykonať kontrolované výpočty a vrátiť štruktúrovaný výsledok bez toho, aby sa LLM pokúšal všetko vyjadriť jedným SQL dotazom.

Router prijíma otázku používateľa, schému databázy a kontext aktuálnej relácie. Jeho výstupom je jednoduché štruktúrované rozhodnutie, napríklad `database` alebo `data_tools`, spolu so stručným dôvodom rozhodnutia. Implementácia kombinuje LLM-based klasifikáciu zámeru podľa promptu a schémy s preddefinovanými pravidlami pre typické dátové úlohy. Najprv sa vyhodnotia bezpečné deterministické pravidlá a LLM sa použije pri nejednoznačných prípadoch.

Tok spracovania je nasledovný:

1. Router dostane otázku, databázovú schému a kontext relácie.
2. Vyhodnotí, či je vhodnejšia cesta `database` alebo `data_tools`.
3. Pri výstupe `database` backend spustí existujúcu NL2SQL pipeline.
4. Pri výstupe `data_tools` backend vyberie a spustí vhodný Python analytický modul.
5. Výsledok sa zjednotí do odpovede používateľovi, aby frontend nemusel rozlišovať internú cestu spracovania.

Router rozširuje systém z čisto text-to-SQL nástroja na všeobecnejší analytický orchestrátor. Zároveň ponecháva kontrolu nad rizikovejšími alebo štatisticky citlivými operáciami v preddefinovaných moduloch, čo zlepšuje reprodukovateľnosť a znižuje riziko, že LLM zvolí nesprávny výpočtový postup.

4.2.3 API Endpoints

Endpoint	Metóda	Popis
/api/ask	POST	Hlavný endpoint - otázka → SQL → sumarizácia
/api/generate-sql	POST	Generovanie SQL bez vykonania
/api/query-and-summarize	POST	Vykonanie SQL a sumarizácia
/api/connect-database	POST	Pripojenie k databáze
/api/disconnect-database	POST	Odpojenie databázy
/api/schema	GET	Získanie schémy databázy
/api/ask-chart	POST	Vygenerovanie grafu na základe otázky
/api/new-chat	POST	Vytvorenie novej chat session
/api/cache-stats	GET	Štatistiky Redis cache
/api/history/{session_id}	GET	Získanie histórie chat session
/api/sessions	GET	Zoznam sessions používateľa
/api/mongo/sessions	GET	Zoznam sessions z MongoDB
/api/database/status	GET	Stav pripojenia k databáze

Tabuľka 4: REST API endpoints

4.2.4 Bezpečnostné mechanizmy

Systém implementuje viacúrovňové zabezpečenie pre READ-ONLY prístup:

1. **LLM Prompt** - Systémový prompt zakazuje generovanie modifikačných príkazov
2. **Regex validácia** - Kontrola nebezpečných kľúčových slov (INSERT, UPDATE, DELETE, DROP...)
3. **SQL Session** - Nastavenie SET TRANSACTION READ ONLY na úrovni databázy

4.2.5 Frontend komponenty

Komponent	Popis
HomePage	Hlavná stránka aplikácie, správa stavu pripojenia
Chat	Chat rozhranie pre komunikáciu s analytikom
SideBar	Bočný panel s ovládacími prvkami
DatabaseModal	Modal pre zadanie pripojovacích údajov k data-báze
InputQuestion	Vstupné pole pre zadávanie otázok
UserMessage	Zobrazenie správ v chate

Tabuľka 5: React komponenty frontendu

5 Frontendová časť aplikácie

5.1 Štruktúra projektu

Frontendová časť aplikácie je implementovaná pomocou knižnice React v kombinácii s nástrojom Tailwind CSS. Projekt je rozdelený do modulov, aby bol prehľadný a komponenty sa dali ľahko znovu použiť. Komponenty sú navrhnuté tak, aby boli čo najviac nezávislé a komunikovali medzi sebou prostredníctvom vlastností (props).

Adresárová štruktúra projektu je nasledovná:

- **components/** - Komponenty, ktoré tvoria používateľské rozhranie, napríklad chat, bočný panel, modálne okná a vstupné polia.
- **hooks/** - Vlastné React hooky, ktoré riešia logiku aplikácie, napríklad pripojenie k databáze.
- **services/** - Funkcie na komunikáciu s backendom.
- **pages/** - Komponenty pre hlavné stránky aplikácie.
- **assets/** - Obrázky, ikony a iné statické súbory.
- **api/** - Nastavenie HTTP klienta pre volania na backend.

5.2 Architektúra a dátový tok

Architektúra frontendu je postavená na jednosmernom toku dát zhora nadol, typickým pre aplikácie v Reacte. Centrálnym bodom aplikácie je hlavný kontajner, komponent `HomePage.jsx`. Tento komponent zabezpečuje stav celej aplikácie a koordinuje komunikáciu medzi vnorenými modulmi.

Dáta a stavy (napr. informácia, či je aplikácia pripojená k databáze, alebo aká je aktívna relácia) sa posúvajú do podriadených komponentov pomocou takzvaných *props* (vlastností). Ak vnorený komponent, akým je napríklad `SideBar`, zaznamená akciu používateľa, nevymieňa si túto informáciu priamo s chatom, ale vyvolá príslušnú callback funkciu smerom nahor do `HomePage`. Tam sa stav vyhodnotí a podľa potreby sa zmení zobrazenie alebo sa vyvolá varovné modálne okno na prepnutie databázy.

5.3 Správa stavu (State Management)

Aplikácia využíva lokálnu správu stavov, nakoľko zložitosť toku dát plne pokrývajú natívne hooky `useState` a `useEffect`. Globálna logika pripojenia k databáze je pre lepšiu prehľadnosť abstrahovaná v rámci vlastného hooku `useDatabase`. Tento hook poskytuje aplikácii zoznam metód (`connect`, `disconnect`) a stavových indikátorov (`loading`, `error`, `sessionId`), vďaka čomu sa nezahŕcaje samotná prezentačná vrstva zložitými operáciami.

V komponente `HomePage` sa tiež eviduje kontext konverzácií. Ak dôjde k načítaniu hlavnej stránky, spustí sa asynchrónna operácia (`getDatabaseStatus`), ktorá overí pripojenie, ako ukazuje nasledujúci blok kódu:

```
1  useEffect(() => {  
2    const checkConnection = async () => {  
3      try {
```

```

4      const res = await getDatabaseStatus();
5      setIsConnected(res.connected);
6    } catch (err) {
7      setIsConnected(false);
8    }
9  };
10  checkConnection();
11 }, []);

```

5.4 Komunikácia s backendom

Celá sieťová vrstva komunikujúca so serverom je centralizovaná do modulu `databaseService.js`. Pre spracovanie HTTP požiadaviek je využívaná knižnica **Axios**.

Aplikácia pracuje primárne s komunikáciou vo formáte JSON. Servisné funkcie ako `listMongoSessions` sťahujú údaje z API a starajú sa o základné vyhodnotenie odpovede zo servera, kým do komponentov vracajú už priamo štruktúrované objekty s dátami, ktoré vie React iterovať a vykresliť.

```

1  import api from "../api";
2
3  export const connectDatabase = async (data) => {
4    const response = await api.post("/api/connect-database", data);
5    return response.data;
6  };
7
8  export const listMongoSessions = async () => {
9    const response = await api.get("/api/mongo/sessions");
10   if (!response.data) {
11     throw new Error("No data returned");
12   }
13   return response.data;
14 };

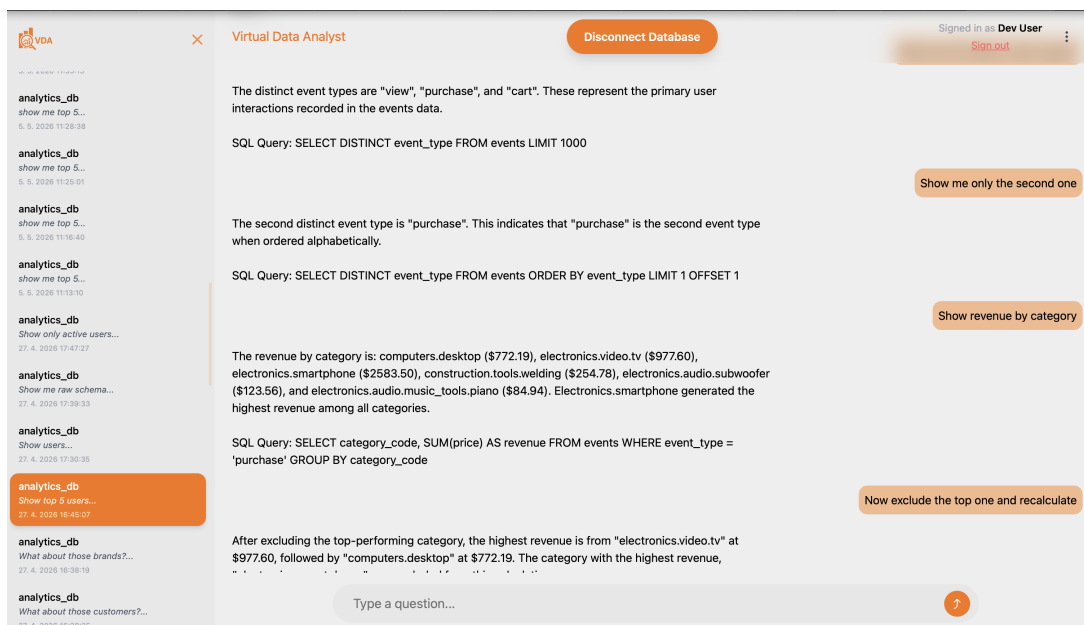
```

Samotný dopyt používateľa je odosielaný prostredníctvom metódy `askQuestion`, ktorá zabezpečuje prenos otázky do backendu pre LLM spracovanie a spätne navracia textovú odpoveď systému, interpretovaný SQL dotaz a počet výsledných riadkov (`row_count`).

5.5 Rozbor kľúčových UI komponentov

5.5.1 Komponent Chat

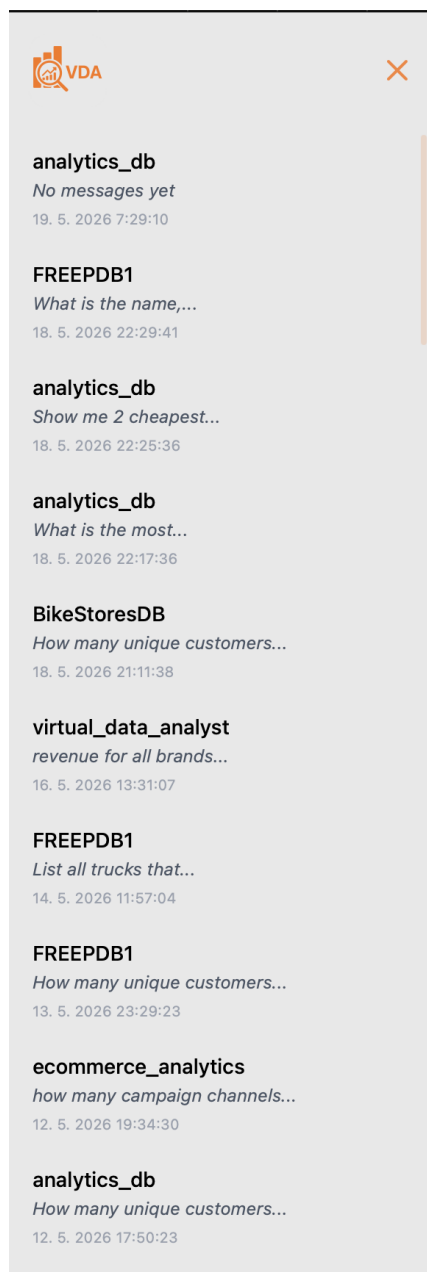
Komponent `Chat.jsx` predstavuje samotné interaktívne jadro celého systému. Zodpovedá za zobrazovanie zoznamu správ a udržiavanie histórie konkrétneho vlákna. Po prijatí reťazca `sessionId` od nadradeného komponentu stiahne prostredníctvom `useEffect` obsah príslušnej histórie cez funkciu `getSessionHistory`. Počas odosielania novej otázky asynchrónne zablokuje zobrazenie, zapíše do poľa vizuálnu informáciu o načítavaní odpovede a po prijatí dát od virtuálneho analytika ju nahradí kompletnou sumarizáciou, ktorej súčasťou je aj vykonaný SQL dotaz.



Obr. 3: Rozhranie komponentu Chat a interakcia používateľa so systémom

5.5.2 Komponent SideBar (Navigačný panel)

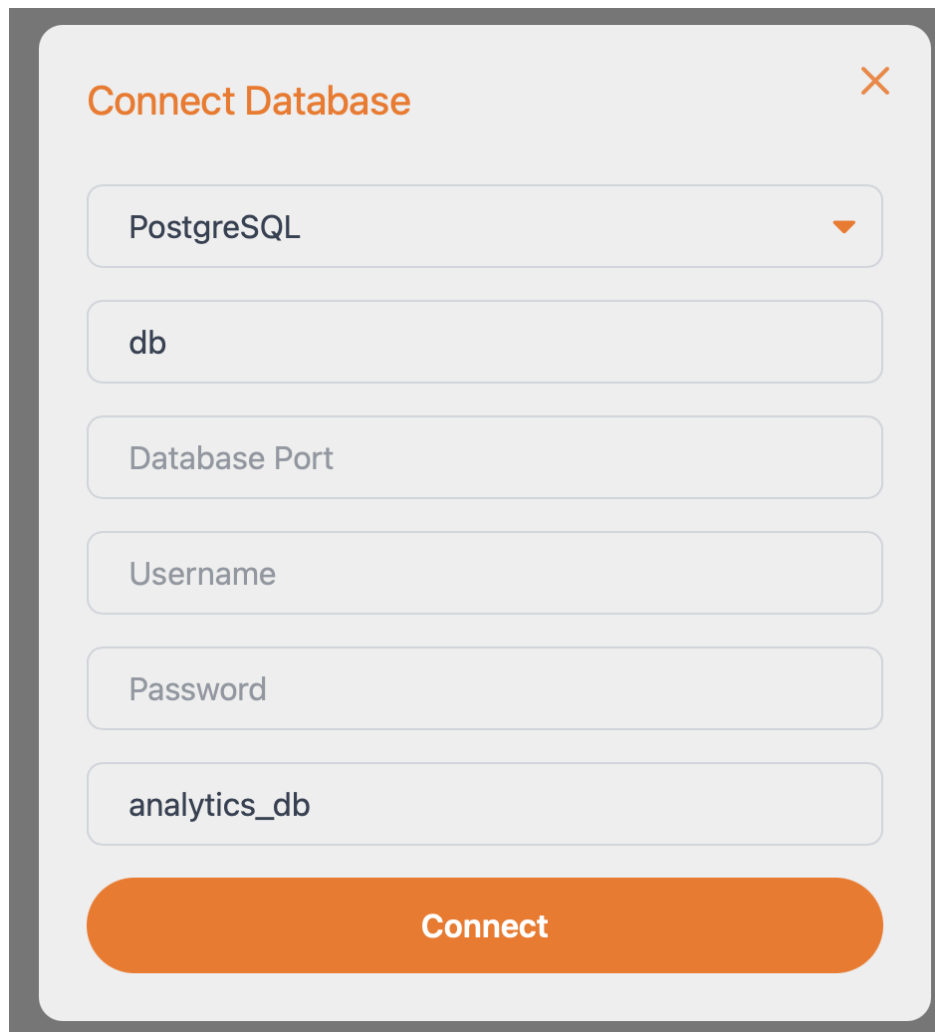
`SideBar.jsx` zastrešuje históriu používateľových sedení (sessions) čerpaných z databázy MongoDB. Prostredníctvom hooku `useEffect` získava zoznam konverzácií a transformuje tieto surové dáta na zoznam histórie chatov (`HistoryBox`). Kľúčovým aspektom Sidebar komponentu je, že po kliknutí na prvok histórie deleguje kompletný objekt dát o vybranej relácii smerom nadol do aplikácie. Ten je následne využitý v nadradenom komponente pre plynulé prepnutie chatu alebo zobrazenie varovania v prípade nekompatibility databáz.



Obr. 4: Zoznam histórie konverzácií v komponente SideBar

5.5.3 Komponent DatabaseModal

`DatabaseModal.jsx` obstaráva modálne okno na manuálne alebo automatizované pripojenie k vybranej databáze. Modal je navrhnutý pomocou značky `<form>`, vďaka čomu systém automaticky zachytáva odosielanie formulára natívnym stlačením klávesy **Enter** z textového poľa. Modul taktiež disponuje pokročilou logikou pre *predvyplnenie dát* (pre-fill). Ak prijme parameter `prefillData` (napr. po výbere inej relácie), dokáže si dynamicky prevziať známe atribúty (`db_type`, `db_host`, `db_name`) a vložiť ich do stavu (`state`). Týmto krokom sa minimalizuje interakcia potrebná pre opakované prepájanie k známym databázam.

A modal window titled "Connect Database" with a close button (X) in the top right corner. It contains several input fields: a dropdown menu currently showing "PostgreSQL", a text field with "db", a text field with "Database Port", a text field with "Username", a text field with "Password", and a text field with "analytics_db". At the bottom is a large orange button labeled "Connect".

Connect Database

PostgreSQL

db

Database Port

Username

Password

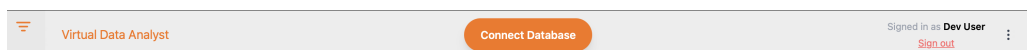
analytics_db

Connect

Obr. 5: Rozhranie modálneho okna pre nadviazanie spojenia s databázou

5.5.4 Komponent ChatHeader

Vrchná informačná a navigačná lišta reprezentuje aktuálne prihláseného používateľa a tlačidlo na pripojenie a odpojenie od databázy.



Obr. 6: Horná lišta ChatHeader

6 Výber a testovanie LLM modelov

Pri návrhu systému bolo kľúčové rozhodnúť, ktoré veľké jazykové modely (LLM) použiť pre jednotlivé úlohy. Keďže systém vyžaduje dva odlišné typy spracovania - generovanie SQL dotazov a sumarizáciu výsledkov - rozhodli sme sa pre špecializovaný prístup s využitím rôznych modelov.

6.1 Metodológia testovania

Pre objektívne porovnanie výkonnosti LLM modelov sme vytvorili testovací skript, ktorý automatizovane vyhodnocoval schopnosti modelov riešiť SQL problémy. Ako zdroj testovacích úloh sme použili problémy z platformy **HackerRank**, ktorá poskytuje široké spektrum SQL výziev rôznej obtiažnosti.

Testovací proces zahŕňal:

1. **Kolekcia problémov** - Výber 50+ SQL problémov z HackerRank (easy, medium, hard)
2. **Automatizované testovanie** - Skript generoval dotazy pomocou každého modelu
3. **Validácia výsledkov** - Porovnanie vygenerovaných dotazov s očakávanými výstupmi
4. **Meranie metrík** - Presnosť, čas odozvy, konzistencia výstupov

6.2 Porovnávané modely

Model	Poskytovateľ
GPT-4.1	OpenAI/Azure
GPT-4o	OpenAI/Azure
GPT-3.5-turbo	OpenAI/Azure
Gemini 2.5 Flash	Google
Gemini 2.5 Pro	Google
Claude 4 Sonnet	Anthropic

Tabuľka 6: Porovnávané LLM modely a ich testované úlohy

6.3 Výsledky testovania - SQL generácia

Pre generovanie SQL dotazov sme testovali modely na nasledujúcich kritériách:

- **Syntaktická správnosť** - Percentuálna úspešnosť generovania validného SQL
- **Sémantická presnosť** - Či dotaz vracia očakávané výsledky
- **Dodržiavanie pravidiel** - Generovanie len READ-ONLY dotazov
- **Konzistencia** - Stabilita výstupov pri opakovaných dotazoch

GPT-4.1 dosiahol dobré výsledky v syntaktickej aj sémantickej presnosti, pričom dosiahol aj dodržiavanie READ-ONLY pravidiel. Napriek vyššej latencii a mierne vyššej cene sme ho zvolili pre kritickú úlohu generovania SQL.

6.4 Výsledky testovania - Sumarizácia

Pre sumarizáciu výsledkov sme hodnotili:

- **Zrozumiteľnosť** - Kvalita textu pre netechnických používateľov
- **Faktická správnosť** - Presnosť interpretácie dát
- **Rýchlosť** - Čas potrebný na generovanie odpovede
- **Cena** - Náklady na API volania

Gemini 2.5 Flash ponúkol optimálny pomer kvality, rýchlosti a ceny. Pri minimálnom rozdiely v kvalite oproti drahším modelom poskytuje výrazne nižšiu latenciu, náklady a veľké kontextové okno.

6.5 Záver výberu modelov

Na základe testovania sme sa rozhodli pre kombináciu:

- **GPT-4.1 (Azure)** pre generovanie SQL: dobrá presnosť a spoľahlivosť pri dodržiavaní bezpečnostných pravidiel
- **Gemini 2.5 Flash (Vertex AI)** pre sumarizáciu: optimálny pomer veľkosť/cena-/rýchlosť

7 Použité technológie - detailný prehľad

7.1 Backend technológie

Python 3.12+

Hlavný programovací jazyk aplikácie. Python je interpretovaný, vysokoúrovňový jazyk so silným ekosystémom knižníc pre dátovú vedu, strojové učenie a webový vývoj. Verzia 3.12 prináša výrazné zlepšenia výkonu interpretera a rozšírenú podporu asynchrónneho programovania. Vďaka jednoduchej syntaxi a rozsiahlej komunite je Python de facto štandardom pre aplikácie využívajúce LLM modely a AI integrácie. Jeho dynamická typovosť je dopĺňaná nástrojmi pre statickú analýzu, čo umožňuje kombináciu rýchlosti vývoja so spoľahlivosťou kódu.

FastAPI

Moderný, asynchrónny webový framework pre budovanie REST API v Pythone. FastAPI je postavený na štandarde ASGI a knižnici Starlette, čo mu umožňuje natívne spracovávať asynchrónne požiadavky s vysokou priepustnosťou. Automaticky generuje OpenAPI dokumentáciu na základe typových anotácií, čím výrazne znižuje objem manuálnej dokumentácie. Integrácia s knižnicou Pydantic zabezpečuje automatickú validáciu vstupných a výstupných dát. FastAPI patrí medzi najrýchlejšie Python frameworky pre webové API, čo ho robí vhodným pre aplikácie s požiadavkami na nízku latenciu.

SQLAlchemy

Komplexná knižnica pre prácu s relačnými databázami v Pythone, fungujúca ako ORM (Object-Relational Mapping) aj ako nástroj pre priame SQL operácie. Poskytuje jednotné rozhranie pre komunikáciu s rôznymi databázovými systémami - PostgreSQL, MySQL, MSSQL, Oracle a ďalšími - prostredníctvom systému dialektov. SQLAlchemy Inspector umožňuje introspekciu databázových metadát, čo aplikácia využíva na dynamické načítavanie schémy. Connection pooling zabudovaný v SQLAlchemy efektívne spravuje životný cyklus databázových spojení a znižuje réžiu pri opakovaných pripojeniach.

OpenAI SDK

Oficiálna klientská knižnica pre komunikáciu s OpenAI API a kompatibilnými LLM endpointami. SDK poskytuje jednoduché rozhranie pre volanie modelov cez štandardizované Chat Completions API, pričom podporuje synchronné aj asynchrónne volania. Knižnica zabezpečuje správu HTTP spojení, serializáciu požiadaviek a deserializáciu odpovedí vrátane štruktúrovaných výstupov. V projekte sa používa nielen pre OpenAI modely (GPT-4.1), ale aj pre prístup k modelom Google Gemini cez kompatibilný endpoint, čo umožňuje jednotné rozhranie pre viacero poskytovateľov LLM.

Pydantic

Knižnica pre validáciu dát a správu nastavení v Pythone, postavená na typových anotáciách. Pydantic automaticky overuje typy, formáty a obmedzenia vstupných dát pri ich spracovaní, čím predchádza chybám spôsobeným neplatnými vstupmi. V kombinácii s FastAPI definuje schémy pre API požiadavky a odpovede, ktoré slúžia aj ako podklad

pre automatickú OpenAPI dokumentáciu. Verzia 2 priniesla výrazné zlepšenie výkonu validácie a rozšírenú podporu pre komplexné dátové modely. Pydantic Settings poskytuje typovo-bezpečné načítanie konfigurácie z premenných prostredia.

uv

Moderný správca Python balíčkov a virtuálnych prostredí implementovaný v jazyku Rust. Na rozdiel od tradičného nástroja pip je uv výrazne rýchlejší pri inštalácii závislostí vďaka paralelizácii a agresívnemu cachovaniu. Podporuje správu závislostí cez súbor `pyproject.toml` a deterministické zamykanie verzií cez `uv.lock`, čo zaručuje reprodukovateľné prostredie vo vývoji aj v CI/CD pipeline. uv tiež zjednodušuje správu Python verzií a virtuálnych prostredí v jednom nástroji.

7.2 Frontend technológie

React

JavaScriptová knižnica pre budovanie používateľských rozhraní. React zavádza komponentový model vývoja, kde sa UI skladá z nezávislých, znovupoužiteľných komponentov s vlastným stavom a logikou. Virtuálny DOM zabezpečuje efektívne aktualizácie rozhrania tým, že minimalizuje priame manipulácie s reálnym DOM stromom. Systém hookov (`useState`, `useEffect` a ďalšie) umožňuje správu stavu a vedľajších efektov vo funkcionálnych komponentoch bez potreby tried. React je v súčasnosti jednou z najrozšírenejších knižníc pre tvorbu webových aplikácií s rozsiahlym ekosystémom doplnkov.

Vite

Moderný build nástroj a vývojový server pre JavaScript aplikácie. Vite využíva natívne ES moduly prehliadača počas vývoja, čo eliminuje potrebu bundlovania a dramaticky skraca čas štartu vývojového servera. Pre produkčné buildy používa Rollup, ktorý generuje optimalizované a minifikované výstupy. Hot Module Replacement (HMR) v prostredí Vite funguje takmer okamžite aj pri veľkých projektoch, čo výrazne zrýchľuje iteračný vývoj. Vite je navrhnutý ako nástupca starších nástrojov ako webpack a Create React App.

Tailwind CSS

Utility-first CSS framework, ktorý namiesto preddefinovaných komponentov poskytuje nízkoúrovňové CSS triedy aplikovateľné priamo v HTML. Tento prístup eliminuje potrebu písania vlastných CSS súborov pre väčšinu prípadov a vedie k konzistentnejšiemu vizuálnemu dizajnu. Tailwind obsahuje rozsiahly systém designových tokenov pre farby, medzery, typografiu a ďalšie vlastnosti. Produkčný build automaticky odstraňuje nepoužívané CSS triedy (tree-shaking), čím minimalizuje veľkosť výsledného CSS súboru. Responzívny dizajn je riešený pomocou prefixov breakpointov priamo v triede.

Axios

HTTP klientská knižnica pre JavaScript podporujúca požiadavky z prehliadača aj z prostredia Node.js. Axios poskytuje jednotné Promise-based API pre vykonávanie HTTP požiadaviek s automatickou serializáciou JSON dát. Podporuje interceptory pre globálne

spracovanie požiadaviek a odpovedí, čo uľahčuje implementáciu autentifikácie a centralizovaného spracovania chýb. Na rozdiel od natívneho `fetch` API automaticky odmieta Promise pri HTTP chybových stavoch (4xx, 5xx) a poskytuje detailnejšie chybové objekty. V projekte je nakonfigurovaný centrálny Axios klient so základnou URL adresou backendu a spoločnými hlavičkami.

7.3 Infraštruktúra

Docker

Platforma pre kontajnerizáciu aplikácií, ktorá zabaľuje aplikáciu a všetky jej závislosti do izolovaného, prenosného kontajnera. Kontajnery zdieľajú jadro operačného systému hostiteľa, čo ich robí oveľa ľahšími ako tradičné virtuálne stroje. Dockerfile definuje sériu inštrukcií pre zostavenie obrazu, čím zabezpečuje plne reprodukovateľné prostredie nezávislé od konfigurácie hostiteľského systému. Docker eliminuje problém „funguje to iba u mňa“ zjednodušuje nasadenie aplikácií na rôzne infraštruktúry. Je základným stavebným kameňom moderných CI/CD pipeline a cloudových nasadení.

Docker Compose

Nástroj pre definovanie a spúšťanie viacerých Docker kontajnerov ako jednej aplikácie pomocou YAML konfiguračného súboru. Docker Compose umožňuje deklaratívne definovať služby, siete a volumes, čo zjednodušuje orchestráciu komplexných aplikácií skladajúcich sa z viacerých komponentov (backend, frontend, databáza, cache). Príkazom `docker compose up` sa spustia všetky služby s ich závislosťami v správnom poradí. V projekte sa Docker Compose využíva nielen pre hlavnú aplikáciu, ale aj pre správu rôznych testovacích databáz, pričom každý databázový systém má vlastný konfiguračný súbor.

Redis

In-memory databáza a cache systém s otvoreným zdrojovým kódom, optimalizovaná pre extrémne rýchly prístup k dátam. Redis ukladá dáta primárne v operačnej pamäti, čo umožňuje časy odozvy v rádoch mikrosekúnd. Podporuje rôzne dátové štruktúry - reťazce, zoznamy, množiny, zoradené množiny a hashe - čo ho robí vhodným pre široké spektrum prípadov použitia od cachovania po správu frontov. TTL (Time-To-Live) mechanizmus umožňuje automatické vyprávanie uložených dát, čo je kľúčové pre udržiavanie aktuálnosti cache. V projekte Redis slúži pre štyri úrovne cachovania a pre ukladanie sémantických embeddingov.

7.4 AI/ML modely

- **GPT-4.1 (Azure)** - Generovanie SQL dotazov
- **Gemini 2.5 Flash (Vertex AI)** - Sumarizácia výsledkov

8 Databázová vrstva

Databázová vrstva predstavuje jednu zo základných súčastí aplikácie. Zabezpečuje pripojenie k používateľovým SQL databázam, vynucovanie read-only prístupu, získavanie schémy a vykonávanie dotazov. Okrem toho systém využíva dve ďalšie databázové technológie - MongoDB pre perzistentnú históriu chatových relácií a Redis pre viacúrovňové cachovanie.

8.1 Podporované databázové systémy

Aplikácia podporuje pripojenie k nasledujúcim relačným databázovým systémom:

- **PostgreSQL** - podpora cez psycopg2 driver,
- **MySQL** - podpora cez pymysql driver,
- **Microsoft SQL Server** - podpora cez pyodbc a odbc driver,
- **Oracle Database** - podpora cez python-oracledb.

Typ databázy používateľ zadáva pri pripájaní. Backend na základe neho zostaví správny connection string a zvolí príslušný SQLAlchemy dialekt. Aplikácia zároveň automaticky konfiguruje bezpečnostné mechanizmy špecifické pre daný dialekt.

8.2 Správa databázového pripojenia

Správa pripojenia je centralizovaná v samostatnej komponente **DatabaseManager**, ktorá zapuzdruje celý životný cyklus spojenia - od inicializácie až po odpojenie. Komponenta je postavená na SQLAlchemy ORM a všetky jej operácie sú implementované asynchrónne, čo umožňuje paralelné spracovanie viacerých požiadaviek bez blokovania aplikácie.

Pre efektívne využívanie zdrojov je použitý **connection pool** - sada trvalých pripojení udržiavaných na pozadí. Pool zabezpečuje automatické obnovenie zlyhajúcich spojení a obmedzuje počet súbežných pripojení k databáze. Pred každým použitím spojenia sa overuje jeho platnosť, čím sa predchádza chybám pri výpadkoch siete.

8.2.1 Získavanie schémy

Po pripojení systém načíta štruktúru databázy priamo z jej systémových metadát. Pre každú tabuľku získa zoznam stĺpcov vrátane ich názvov, dátových typov a informácie o tom, či stĺpec môže obsahovať prázdnu hodnotu. Takto získaná schéma slúži ako kontext pre LLM model - model dostane úplný prehľad tabuliek a stĺpcov, aby mohol generovať syntakticky a sémanticky správne dotazy.

8.2.2 Vykonávanie dotazov

Dotazy sa vykonávajú výhradne v read-only režime. Bezpečnosť je zaistená na troch úrovniach, ktoré sú popísané v nasledujúcej podsekcii. Výsledky dotazu sú vrátené ako štruktúrovaný zoznam záznamov, kde každý záznam obsahuje hodnoty stĺpcov pod ich názvami.

8.3 READ-ONLY prístup a bezpečnosť

Jeden z najdôležitejších aspektov databázovej vrstvy je garantovaný read-only prístup. Ochrana je implementovaná na troch nezávislých úrovniach:

1. **Úroveň jazykového modelu** - systémový prompt pre SQL generátor výslovne zakazuje generovanie akýchkoľvek modifikačných príkazov a povoľuje výlučne príkazy SELECT,
2. **Úroveň regulárnych výrazov** - pred vykonaním sa každý dotaz skontroluje na prítomnosť nebezpečných kľúčových slov (INSERT, UPDATE, DELETE, DROP, CREATE, ALTER, TRUNCATE, GRANT, REVOKE); detekcia je nezávislá od veľkosti písmen a pri nájdení zakázaného slova sa dotaz odmietne,
3. **Úroveň databázovej transakcie** - pre podporované databázové systémy sa read-only mód nastaví priamo na úrovni databázovej transakcie pomocou databázovo špecifického príkazu.

Tento trojúrovňový prístup zabezpečuje, že ani pri kompromitácii jednej vrstvy nedôjde k nežiaducim zmenám v databáze.

8.4 Parametrizácia pripojenia

Aplikácia umožňuje dynamické pripojenie na ľubovoľnú databázu bez zmeny kódu. Používateľ zadáva nasledujúce parametre:

Parameter	Popis
Typ databázy	Dialekt (PostgreSQL, MySQL, SQL Server, Oracle)
Host	Hostname alebo IP adresa servera
Port	Port, na ktorom databázový server počúva
Používateľské meno	Prihlasovacie meno
Heslo	Prístupové heslo
Názov databázy	Meno databázy alebo service name (Oracle)

Tabuľka 7: Parametre pre pripojenie k databáze

Backend tieto parametre skombinuje do connection stringu kompatibilného s príslušným dialektom, inicializuje databázové pripojenie a zároveň vytvorí novú reláciu v MongoDB pre históriu chatu. Pri odpojení sa uzavrie databázové spojenie aj MongoDB session.

8.5 API endpointy pre správu pripojenia

Endpoint	Metóda	Popis
/api/connect-database	POST	Pripojenie k databáze, vytvorenie MongoDB session
/api/disconnect-database	POST	Odpojenie, uzavretie session, vyčistenie cache
/api/schema	GET	Vrátenie aktuálnej schémy databázy

Tabuľka 8: Endpointy pre správu databázového pripojenia

8.6 Testovacie databázy a Docker Compose

Pre vývoj a testovanie aplikácie je pripravených niekoľko samostatných Docker Compose konfigurácií, každá so svojím datasetom:

Konfigurácia	Obsah
PostgreSQL - e-commerce	Zákaznícke správanie v e-commerce prostredí
PostgreSQL - bike store	Predajné dáta obchodu s bicyklami
MySQL - e-commerce	E-commerce analýza v MySQL dialekte
Microsoft SQL Server	Testovacia databáza pre enterprise prostredie
Oracle Database	Testovacia databáza pre korporátne prostredie

Tabuľka 9: Testovacie databázy pre vývoj

Každá konfigurácia obsahuje health check pre monitorovanie dostupnosti, automatickú inicializáciu dát zo SQL skriptov a CSV súborov a persistentné volumes pre ukladanie dát medzi reštartmi kontajnera.

8.7 MongoDB - história chatových relácií

Pre perzistentnú históriu konverzácií systém využíva MongoDB. Každá databázová session má vlastný dokument obsahujúci metadáta o pripojení (typ databázy, host, názov databázy, čas pripojenia a odpojenia) a pole správ konverzácie. Každá správa uchováva rolu (používateľ alebo asistent), textový obsah, prípadný SQL dotaz, počet vrátených riadkov a časovú pečiatku.

Listing 1: štruktúra MongoDB session

```
1 {  
2   "session_id": "uuid",  
3   "user_id": "...",  
4   "db_type": "postgresql",  
5   "db_host": "...",  
6   "db_name": "...",  
7   "connected_at": "datetime",  
8   "disconnected_at": None,  
9   "messages": [  
10
```

```

10     {
11         "role": "user" | "assistant",
12         "content": "...",
13         "sql_query": "...",
14         "row_count": 42,
15         "success": True,
16         "timestamp": "datetime"
17     }
18 ]
19 }

```

Pre rýchly prístup sú vytvorené indexy na identifikátor session a na kombináciu identifikátora používateľa s časom pripojenia, čo umožňuje efektívne načítanie zoznamu relácií zoradených od najnovšej.

História relácií sa využíva v dvoch smeroch: frontend ju zobrazuje používateľovi v sídebare, a backend z nej extrahuje kontext pre LLM model pri generovaní SQL dotazov.

8.8 Zachovanie kontextu medzi otázkami

Počas vývoja backendu sme riešili problém zachovania kontextu medzi používateľom a LLM modelom. Model pôvodne spracovával každú otázku samostatne, čo spôsobovalo problémy pri nadväzujúcich otázkach. Preto bola do aplikácie doplnená funkcionálna pre prácu s históriou konverzácie, vďaka ktorej model dokáže prirodzene nadväzovať na predchádzajúcu komunikáciu v rámci jednej používateľskej relácie.

Pri vytvorení databázového pripojenia backend automaticky vytvorí novú session, ku ktorej sa počas celej komunikácie ukladajú otázky používateľa, odpovede asistenta a vygenerované SQL dotazy. Takto vzniká história konverzácie, s ktorou backend pracuje pri spracovaní ďalších požiadaviek.

8.8.1 Kontext pre SQL generátor

Pri prijatí novej otázky backend načíta aktuálnu session a z histórie správ pripraví kontext pre LLM model. Do kontextu sa posielajú iba najrelevantnejšie informácie - nie celá história, pretože pri dlhších konverzáciách by zbytočne narastal počet tokenov a znižovala sa efektivita spracovania:

- niekoľko posledných správ z konverzácie,
- predchádzajúca otázka používateľa,
- posledný vygenerovaný SQL dotaz.

Tento kontext je odovzdaný SQL generátoru, ktorý ho zahrnie do promptu. Vďaka tomu model rozumie nadväzujúcim otázkam - napríklad:

Ukáž posledné objednávky
teraz len top 10

Model vďaka predchádzajúcemu kontextu chápe, že druhá otázka odkazuje na predchádzajúci SQL dotaz a má ho upraviť aplikovaním dodatočného obmedzenia. Bez zachovania kontextu by model druhú otázku nedokázal správne interpretovať, pretože sama o sebe neobsahuje dostatok informácií na vytvorenie relevantného SQL dotazu.

Prácu s históriou komunikácie možno rozdeliť na dve časti:

1. **Začiatok požiadavky** - backend načíta aktuálnu session, získa históriu správ a pripraví kontext pre LLM model.
2. **Koniec požiadavky** - po úspešnom spracovaní sa do histórie uloží nová správa používateľa, vygenerovaný SQL dotaz a výsledná odpoveď asistenta.

História konverzácie sa priebežne aktualizuje po každej požiadavke, čím má model pri ďalších otázkach k dispozícii informácie o predchádzajúcej komunikácii. Takýto prístup umožňuje prirodzenejšiu komunikáciu medzi používateľom a systémom a zároveň znižuje potrebu opakovania rovnakých informácií pri pokračujúcich otázkach.

Frontend zároveň uchováva identifikátor aktuálnej session pomocou `localStorage`, vďaka čomu zostáva referencia na konverzáciu zachovaná aj po obnovení stránky. Predchádzajúce relácie sa zobrazujú v bočnom paneli aplikácie, čo umožňuje pokračovať v komunikácii bez straty histórie konverzácie.

8.9 Redis - cachovanie

Pre zrýchlenie opakovaných operácií systém využíva Redis ako in-memory cache. Redis sa pripája pri štarte aplikácie v rámci životného cyklu FastAPI. Implementácia je navrhnutá ako *best-effort* vrstva: ak Redis nie je dostupný, backend pokračuje bez cache a vykoná štandardné volania databázy alebo LLM modelov. Všetky cache hodnoty sa ukladajú ako JSON a čítajú sa cez spoločný modul `redis_client.py`, ktorý zapuzdruje operácie `get`, `set`, `incr`, prácu so zoradenými množinami a mazanie podľa vzoru.

Cache kľúče sú odvodené od prefixu `vda` a odtlačku aktuálneho databázového pripojenia. Odtlačok vzniká ako krátky hash connection stringu, takže údaje z jednej databázy sa nemôžu omylom použiť pri práci s inou databázou. Pri otázkach sa navyše používa normalizovaná signatúra otázky, pri SQL výsledkoch hash normalizovaného SQL dotazu a pri sumarizácii aj hash pôvodnej otázky ako kontextu. Pri nadväzujúcich otázkach sa do kľúča pre NL2SQL pridáva aj hash kontextu konverzácie, aby sa rovnaká krátka otázka typu „teraz len top 10“ nevyhodnotila bez predchádzajúceho významu.

Implementovaných je viacero samostatných vrstiev cachovania:

Cache	Obsah	Kľúč	TTL
Schéma	Štruktúra databázy (tabuľky a stĺpce)	prefix + odtlačok databázy	1 hodina
NL2SQL	Vygenerovaný SQL pre danú otázku a kontext	prefix + odtlačok DB + hash signatúry otázky + hash kontextu	7 dní
Výsledky SQL	Výsledky vykonaného dotazu	prefix + odtlačok DB + hash SQL	15 minút
Sumarizácia	Textová sumarizácia výsledkov	prefix + odtlačok DB + hash SQL + hash otázky	15 minút
História chatu	Posledné správy relácie pre rýchle načítanie histórie	prefix + identifikátor session	1 deň

Tabuľka 10: Úrovně Redis cache

Každá cache vrstva funguje na princípe **cache-aside**: najprv sa pokúsi načítať dáta z Redisu, pri cache miss vykoná skutočnú operáciu a výsledok uloží späť do Redisu s nastaveným TTL. Pri schéme to znamená volanie introspekcie databázy, pri NL2SQL volanie generátora dotazov, pri výsledkoch vykonanie SQL nad databázou a pri sumarizácii volanie sumarizačného modelu. Výsledky SQL sa ukladajú iba vtedy, ak počet riadkov nepresiahne nakonfigurovaný limit `cache_max_rows`, ktorý je nastavený na 2000. Tým sa zabraňuje tomu, aby veľké odpovede zbytočne zaplňali Redis.

Cache sa používa aj pri požiadavkách na grafy. Ak si používateľ vyžiada vizualizáciu, backend najskôr zistí zámer grafu a potom môže opätovne použiť cache schémy a cache výsledkov SQL. Samotné dáta pre graf sú teda chránené rovnakým mechanizmom ako bežné výsledky dotazov, pričom hlavička odpovede obsahuje aj stav **X-Cache-Chart-Results**. Tento prístup znižuje duplicitu medzi textovou odpoveďou a vizualizačnou vetvou spracovania.

Pre históriu konverzácie existuje samostatná cache vrstva. Endpoint pre načítanie histórie relácie najprv hľadá záznam v Redise a až pri cache miss načítava dokument z MongoDB. Po každej úspešnej odpovedi sa do cache doplní správa používateľa aj asistenta, pričom sa uchováva najviac posledných 50 správ. MongoDB tak zostáva trvalým zdrojom pravdy, ale opakované otváranie tej istej relácie je rýchlejšie a menej zaťažuje databázu.

8.9.1 Sémantická Q&A cache

Nad štandardným cachovaním je implementovaná sémantická cache pre celé dvojice otázka-odpoveď. Pri každom úspešnom spracovaní otázky sa uloží vektorová reprezentácia (embedding) otázky spolu s SQL dotazom, sumarizáciou a počtom riadkov. Embedding sa vytvára modelom `azure.text-embedding-3-small`. Jednotlivé záznamy sa ukladajú ako samostatné Redis objekty a ich identifikátory sú vedené v zoradenej množine podľa času vytvorenia. Cache je obmedzená na 5000 položiek na databázový odtlačok a pri prekročení limitu sa odstraňujú najstaršie záznamy.

Pri ďalšom dopyte systém porovná embedding novej otázky s uloženými pomocou cosine similarity. Prehľadáva sa najviac 200 najnovších kandidátov a ak podobnosť prekročí nakonfigurovaný prah 0,70, backend vráti uložený SQL dotaz, sumarizáciu a počet riadkov okamžite - bez opätovného generovania SQL, bez sumarizácie a bez spustenia pôvodného databázového dotazu. Sémantická cache navyše overuje zhodu kľúčových slov a číselných hodnôt v otázke. Kľúčové slová majú vlastný minimálny prah podobnosti a pri výpočte finálneho skóre tvoria časť váhy, aby sa zabránilo falošným hitom pri podobne znejúcich otázkach s iným filtrom alebo iným číslom.

Sémantická cache sa nepoužíva vždy. Ak má otázka konverzačný kontext, napríklad predchádzajúcu otázku, predchádzajúci SQL dotaz alebo text histórie, backend ju cielene obíde a do hlavičky nastaví stav **BYPASS**. Je to dôležité najmä pri nadväzujúcich otázkach, ktoré samy o sebe nemajú úplný význam. V takom prípade je bezpečnejšie nechať SQL generátor pracovať s aktuálnym kontextom, než použiť starú odpoveď na základe podobnosti samotného textu otázky.

8.9.2 Cache metriky a diagnostika

Systém sleduje počty cache hitov a missov pre každú vrstvu v Redise. Metriky sa ukladajú pod prefixom `vda:metrics` pre oblasti `schema`, `nl2sql`, `sql_results`, `summary`,

qa_semantic a chat_history. Endpoint /api/cache-stats vracia pre každú oblasť počet hitov, missov, percentuálnu úspešnosť a aj počty aktuálnych Redis kľúčov podľa typu cache. Ak Redis nie je pripojený, endpoint explicitne vracia stav `redis_unavailable`.

Každá odpoveď z API obsahuje HTTP hlavičky s informáciami o stave cache:

- X-Cache-Schema - HIT alebo MISS pre schému databázy,
- X-Cache-NL2SQL - HIT alebo MISS pre SQL generovanie,
- X-Cache-SQL-Results - HIT alebo MISS pre výsledky dotazu,
- X-Cache-Summary - HIT alebo MISS pre sumarizáciu,
- X-Cache-QA-Semantic - HIT alebo MISS pre sémantickú cache,
- X-Cache-Trace - zhrnutie všetkých cache stavov v jednej hlavičke.

Tieto hlavičky umožňujú rýchlu diagnostiku výkonu systému priamo z HTTP klienta alebo vývojárskych nástrojov prehliadača.

9 CI/CD a nasadenie

Projekt využíva priebežnú integráciu cez GitHub Actions. Workflow je definovaný v súbore `.github/workflows/ci.yaml` a spúšťa sa pri pushi do vetvy `main` aj pri pull requestoch. CI pipeline je rozdelená na dve nezávislé časti: backendové kontroly a frontendový lint. Vďaka tomu je možné rýchlo identifikovať, či zmena porušila serverovú časť, klientsku časť alebo iba štýlové pravidlá kódu.

9.1 Backend CI

Backend job beží na prostredí `ubuntu-latest` s Pythonom 3.12. Závislosti sa inštalujú pomocou nástroja `uv` príkazom `uv sync --extra dev --locked`, čím sa používa zamknutý súbor `uv.lock` a zabezpečuje sa reprodukovateľné prostredie. Následne sa spúšťajú štyri hlavné kontroly:

- **Ruff lint** - statická kontrola Python kódu podľa pravidiel v `pyproject.toml`.
- **ty type check** - statická kontrola typov pre balík `app`, testy a vstupný súbor `main.py`.
- **Bandit security lint** - bezpečnostná analýza Python kódu, ktorá hľadá rizikové konštrukcie.
- **pytest** - spustenie automatizovaných testov v adresári `backend/tests`.

Testy overujú hlavné API scenáre vrátane generovania SQL, spracovania otázky cez `/api/ask`, odpovedí s históriou chatu, generovania grafu, pripojenia a odpojenia databázy, načítania schémy a práce s MongoDB sessions. V testoch sú Redis, MongoDB a sémantická cache patchované tak, aby boli testy izolované od reálnej infraštruktúry a dali sa spoľahlivo spúšťať v CI prostredí.

9.2 Frontend CI

Frontend job používa Node.js 20 a cache pre `npm` závislosti podľa frontendového lockfile súboru. Inštalácia prebieha cez `npm ci`, čo je vhodné pre CI prostredie, pretože rešpektuje lockfile a neprepisuje verzie balíkov. Po inštalácii sa spúšťa `npm run lint`, teda ESLint kontrola nad súbormi `.js` a `.jsx`. Konfigurácia lintu zakazuje warningy cez `--max-warnings 0`, takže aj menej závažné porušenia štýlu musia byť vyriešené pred zlúčením zmeny.

9.3 CD na Azure VM

Kontinuálne nasadenie nadväzuje na rovnaké kontroly ako CI. Pred deployom sa najprv spustí linting, typová kontrola, bezpečnostná analýza a testy. Až po ich úspešnom dokončení CD job zostaví alebo stiahne kontajnerové obrazy a nasadí ich na Azure VM. Nasadenie tak nepreskakuje validačné kroky, ktoré chránia vetvu `main`.

Produkčné nasadenie je kontajnerové cez `docker-compose.prod.yml`. Táto konfigurácia obsahuje frontendový kontajner dostupný na porte 80 a backendový kontajner s FastAPI aplikáciou na porte 8000, ktorý má vlastný health check cez endpoint

`/health`. Backend načítava produkčné premenné prostredia zo súboru `.env.prod`, používa politiku `restart: unless-stopped` a frontend čaká na zdravý stav backendu. CD pipeline na Azure VM vykonáva kroky ako prihlásenie na server cez SSH, aktualizácia zdrojového kódu alebo stiahnutie nových obrazov, spustenie `docker compose -f docker-compose.prod.yml up -d --build` a kontrola zdravotného stavu služieb po nasadení.

Takýto model nasadenia je jednoduchší než plnohodnotná orchestrácia v Kubernetes, ale pre tímový projekt poskytuje dostatočnú mieru automatizácie. Azure VM slúži ako stabilný hosťiteľ kontajnerov, Docker Compose zabezpečuje spustenie viacerých služieb v správnom poradí a health checky umožňujú rýchlo zistiť, či nová verzia aplikácie po deployi korektne beží.

Použitie nástrojov umelej inteligencie

Pri vypracovaní projektu sme využili nasledujúce AI nástroje ako podporné prostriedky:

- **Claude Opus 4.5 (Anthropic, 2025):** Využitý pri návrhu architektúry systému, code review a optimalizácii kódu.
- **Claude Sonnet 4.5 (Anthropic, 2025):** Využitý pri návrhu architektúry systému, code review a optimalizácii kódu. Pomáhal s formuláciou systémových promptov pre LLM komponenty.
- **Claude Opus 4.6 (Anthropic, 2026):** Využitý pri návrhu architektúry systému, code review a optimalizácii kódu.
- **Claude Sonnet 4.6 (Anthropic, 2026):** Využitý pri návrhu architektúry systému, code review a optimalizácii kódu. Pomáhal s formuláciou systémových promptov pre LLM komponenty.
- **GPT-4.1 (OpenAI/Azure, 2025):** Integrovaný priamo v systéme ako Query Generator pre transformáciu prirodzeného jazyka na SQL dotazy.
- **Gemini 2.5 Flash (Google, 2025):** Integrovaný v systéme ako Response Summarizer pre generovanie zrozumiteľných textových sumarizácií výsledkov dotazov.

Zdroje

1. FastAPI Documentation. (2025). *FastAPI - Modern, Fast Web Framework*.
<https://fastapi.tiangolo.com/>
2. React Documentation. (2025). *React - A JavaScript Library for Building User Interfaces*.
<https://react.dev/>
3. OpenAI. (2025). *GPT-4 API Documentation*.
<https://platform.openai.com/docs/>
4. Google Cloud. (2025). *Vertex AI - Gemini API*.
<https://cloud.google.com/vertex-ai/docs/generative-ai/>
5. PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*.
<https://www.postgresql.org/docs/>
6. SQLAlchemy Documentation. (2025). *SQLAlchemy - The Database Toolkit for Python*.
<https://docs.sqlalchemy.org/>
7. Docker Documentation. (2025). *Docker Docs*.
<https://docs.docker.com/>
8. Redis Documentation. (2025). *Redis Documentation*.
<https://redis.io/documentation>
9. Oracle Database Documentation. (2026). *Oracle Database Documentation*. <https://docs.oracle.com/en/database/>
10. Microsoft SQL server Documentation. (2026). *SQL Server technical documentation*.
<https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver17>
11. Medium (2023). *Building Fast and Robust APIs with FastAPI and SQL Databases*.
<https://medium.com/towards-data-engineering/fastapi-with-sql-1c7852ccbf21>
12. Budibase. (2023). *How to Integrate Multiple Databases*.
<https://budibase.com/blog/data/how-to-integrate-multiple-databases/>
13. Vite Documentation. (2025). *Vite - Next Generation Frontend Tooling*.
<https://vitejs.dev/>
14. Tailwind CSS Documentation. (2025). *Tailwind CSS - A Utility-First CSS Framework*.
<https://tailwindcss.com/docs>
15. Axios Documentation. (2025). *Axios - Promise Based HTTP Client*.
<https://axios-http.com/docs/intro>
16. React Hooks Documentation. (2025). *React Hooks - State and Lifecycle in Functional Components*.
<https://react.dev/reference/react/hooks>

17. GitHub Docs. (2026). *About workflows in GitHub Actions*.
<https://docs.github.com/actions/using-workflows/about-workflows>
18. Docker Documentation. (2026). *Docker Compose*.
<https://docs.docker.com/compose/>
19. Microsoft Learn. (2026). *Quickstart: Create a Linux virtual machine in the Azure portal*.
<https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal>
20. Microsoft Learn. (2026). *Introduction to Azure Container Registry*.
<https://learn.microsoft.com/en-us/azure/container-registry/container-registry-intro>
21. Redis Documentation. (2026). *Redis documentation*.
<https://redis.io/docs/latest/>
22. LangChain Documentation. (2026). *Router*.
<https://docs.langchain.com/oss/python/langchain/multi-agent/router>
23. pytest Documentation. (2026). *pytest documentation*.
<https://docs.pytest.org/en/stable/>
24. Astral. (2026). *The Ruff Linter*.
<https://docs.astral.sh/ruff/linter/>
25. Astral. (2026). *uv documentation*.
<https://docs.astral.sh/uv/>
26. Astral. (2026). *ty type checking*.
<https://docs.astral.sh/ty/type-checking/>
27. Bandit Documentation. (2026). *Bandit documentation*.
<https://bandit.readthedocs.io/en/latest/>
28. ESLint. (2026). *Getting Started with ESLint*.
<https://eslint.org/docs/latest/use/getting-started>
29. Brown, T., et al. (2020). *Language Models are Few-Shot Learners*.
<https://arxiv.org/abs/2005.14165>
30. Rajkumar, N., et al. (2022). *Evaluating the Text-to-SQL Capabilities of Large Language Models*.
<https://arxiv.org/abs/2204.00498>
31. Pourreza, M., & Rafiei, D. (2024). *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL*.
<https://arxiv.org/abs/2304.11015>